

[ISA5810 DM2026] Lab 1 – Environment Setup

Hi everyone,

The lab material and assignment submission space on NTU COOL will be available starting **September 28 (Monday)** at **9:00 AM**.

The Lab tutorial videos will be launched on the [YouTube Channel](#) before **September 28**.

We strongly recommend setting up the environment on your personal laptop **before** the lab session so you can follow along smoothly. If you run into issues, email the TAs or attend TA sessions on weekdays.

This guide has **two parts**:

Part	When	Where you work
Part A — Environment check	Now (before or after Sept 28)	DM2026-Lab1-Announcement setup repo
Part B — Lab work & submission	After the main lab repo is announced	Your fork of DM2026-Lab1-Exercise (main lab repo)

Where to run the steps: If the main lab repo is **not** released yet, do Part A in the folder where you cloned **DM2026-Lab1-Announcement**. Once **DM2026-Lab1-Exercise** is available, repeat the environment steps (Python, [uv sync](#), API keys, Test-Env) in **that** repo’s root, then continue with Part B.

How to run the lab: Local setup only on your laptop with [uv](#) + Jupyter or VS Code/Cursor/Antigravity. The agentic notebooks require local Jupyter (chat widgets do not work in cloud notebook runtimes).

System requirements

- **Git** and a **GitHub** account. [Tutorial on Git, GitHub and GitHub Desktop](#)
- **uv** (installs and manages **Python 3.11** and the project virtual environment)
- **Jupyter** (installed by [uv sync](#); use VS Code/Cursor/Antigravity or [jupyter lab](#) in the browser).
IMPORTANT NOTE: Using [Jupyter Lab](#) for the [Agentic Section](#) of the lab is extremely recommended over some bugs that have been observed with VS Code / Cursor / Antigravity when running the widget to chat with the agent.
- [VS Code](#), [Cursor](#), or [Antigravity](#) (optional but convenient). Click in the links to download and install the software.

The course targets **Python 3.11** (any patch release — the exact build depends on your OS; see [uv python list](#)). The announcement repo includes [pyproject.toml](#), [uv.lock](#), [requirements.txt](#), and [.python-version \(3.11\)](#) so **package** versions match ours; you do not need to match a specific patch number.

Part A — Environment check (DM2026-Lab1-Announcement)

Do this in the **announcement** repo first, even before the main lab materials are released.

A1. GitHub account and Git

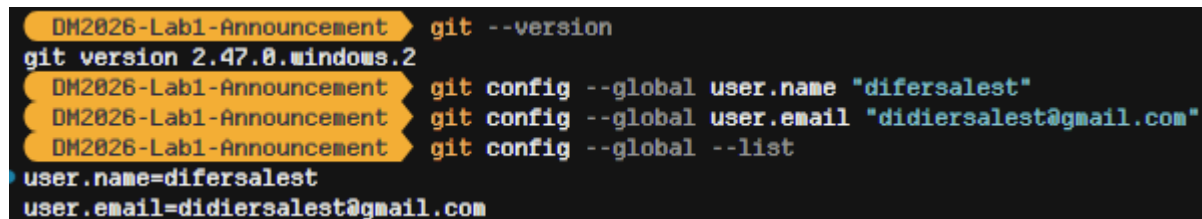
Sign up: <https://github.com/>

Install Git:

- **Windows:** <https://gitforwindows.org/>
- **Linux:** `sudo apt install git-all`
- **macOS:** `brew install git`

Verify and configure (use your GitHub username and email):

```
git --version
git config --global user.name "YOUR_USERNAME"
git config --global user.email "your_email@example.com"
```

A terminal window with a dark background and yellow prompt text. The commands and their outputs are as follows:
1. Prompt: DM2026-Lab1-Announcement, Command: git --version, Output: git version 2.47.0.windows.2
2. Prompt: DM2026-Lab1-Announcement, Command: git config --global user.name "difersalest", Output: (no output)
3. Prompt: DM2026-Lab1-Announcement, Command: git config --global user.email "didiersalest@gmail.com", Output: (no output)
4. Prompt: DM2026-Lab1-Announcement, Command: git config --global --list, Output: user.name=difersalest, user.email=didiersalest@gmail.com

If you prefer a GUI (Graphic User Interface), [GitHub Desktop](#) works too. [Check the GitHub Desktop tutorial in this video](#)

A2. Install uv

In Terminal or PowerShell:

```
pip install uv
uv --version
```

On Windows you can also use the installer from <https://docs.astral.sh/uv/getting-started/installation/>.

A3. Clone the announcement repo

Choose a folder for course work, then clone the **setup** repository (not the main lab repo yet):

```
cd <yourpath>
mkdir DM2026Labs
cd DM2026Labs
git clone https://github.com/difersalest/DM2026-Lab1-Announcement.git
cd DM2026-Lab1-Announcement
```

```

pwsh in DM2026-Lab1-Annou X + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

didif cd documents
documents mkdir DM2026Labs

Directory: C:\Users\didif\documents

Mode                LastWriteTime         Length Name
----                -
d-----          9/25/2026   9:09 PM             DM2026Labs

documents cd DM2026Labs
DM2026Labs git clone https://github.com/difersalest/DM2026-Lab1-Announcement.git
Cloning into 'DM2026-Lab1-Announcement'...
git: 'credential-manager-core' is not a git command. See 'git --help'.
remote: Enumerating objects: 166, done.
remote: Counting objects: 100% (166/166), done.
remote: Compressing objects: 100% (113/113), done.
remote: Total 166 (delta 46), reused 164 (delta 44), pack-reused 0 (from 0)
Receiving objects: 100% (166/166), 6.74 MiB | 11.36 MiB/s, done.
Resolving deltas: 100% (46/46), done.
DM2026Labs cd DM2026-Lab1-Announcement
DM2026-Lab1-Announcement dir

Directory: C:\Users\didif\documents\DM2026Labs\DM2026-Lab1-Announcement

Mode                LastWriteTime         Length Name
----                -
d-----          9/25/2026   9:10 PM             config
d-----          9/25/2026   9:10 PM             img
-a-----          9/25/2026   50          .gitattributes
-a-----          9/25/2026  2546        .gitignore
-a-----          9/25/2026    9          .python-version
-a-----          9/25/2026 31431      DM2026-Lab1-Test-Env.ipynb
-a-----          9/25/2026   700        pyproject.toml
-a-----          9/25/2026 13976      README.md
-a-----          9/25/2026   709        requirements.txt
-a-----          9/25/2026 481549     uv.lock

didif DM2026-Lab1-Announcement main |

```

A4. Python 3.11 and dependencies

From the **root** of `DM2026-Lab1-Announcement` (where `pyproject.toml`, `uv.lock`, and `.python-version` live):

Install a **3.11** interpreter once (uv downloads and manages it; the patch version is chosen for your platform):

```
uv python install 3.11
```

To see which 3.11 builds uv can install on your machine:

```
uv python list | findstr 3.11
```

On macOS/Linux, use `grep` instead of `findstr`:

```
uv python list | grep 3.11
```

A terminal window showing the command 'uv python install 3.11' and its output. The output indicates that Python 3.11.13 was installed in 46 seconds. It also shows the path to the installed Python executable: 'c:\python-3.11.13-windows-x86_64-none\python3.11.exe'. A warning message states that this path is not on the user's PATH and provides instructions on how to update the shell's PATH environment variable.

```
DM2026-Lab1-Announcement uv python install 3.11
• Installed Python 3.11.13 in 46.16s
• cpython-3.11.13-windows-x86_64-none (python3.11.exe)
warning: 'C:\Users\didif\local\bin' is not on your PATH. To use installed Python executables, run '$env:PATH = "C:\Users\didif\local\bin;$env:PATH"' or 'uv python update-shell'.
```

Create the virtual environment and install all packages at the pinned versions:

```
uv sync
```

`uv sync` reads `.python-version` (3.11) and uses the 3.11 interpreter you installed above. This creates `.venv` in the project folder and installs everything from `uv.lock`. **Let it finish with no errors** before opening any notebook. A partial install often breaks later notebooks (especially `ipywidgets` for the agent chat).

```
DM2026-Lab1-Announcement uv sync
• Using CPython 3.11.13
Creating virtual environment at: .venv
Resolved 207 packages in 1ms
Installed 208 packages in 4.58s
+ aiohappyeyeballs==2.6.1
+ aiohttp==3.12.15
+ aiosignal==1.4.0
+ alabaster==0.7.16
+ annotated-types==0.8.0
+ anyio==4.10.0
+ argon2-cffi==25.1.0
+ argon2-cffi-bindings==25.1.0
+ arrow==1.3.0
+ asttokens==3.0.0
+ async-lru==2.0.5
+ attrs==25.3.0
+ babel==2.17.0
+ beautifulsoup4==4.13.4
+ bleach==6.2.0
+ certifi==2025.8.3
+ cffi==2.1.1
+ charset-normalizer==3.4.3
+ choreographer==1.4.0
+ click==8.1.8
+ colorama==0.4.6
+ comm==0.2.3
+ contourpy==1.3.0
+ cramjam==2.11.0
+ cryptography==50.0.1
+ cycler==0.12.1
+ debugpy==1.8.16
+ decorator==5.2.1
+ defusedxml==0.7.1
+ deprecated==1.2.18
+ discord-py==2.6.0
+ distro==1.9.0
+ docutils==0.21.2
+ executing==2.2.0
+ fastjsonschema==2.21.2
+ fastparquet==2024.11.0
+ filetype==1.2.0
+ fonttools==4.59.1
+ fqdn==1.5.1
+ frozenlist==1.7.0
+ fsspec==2025.7.0
```

Without uv (pip fallback only): use any **Python 3.11.x** on your PATH, then from the repo root:

```
pip install -r requirements.txt
```

Verify Python version:

```
uv run python --version
```

Expected: **Python 3.11.x** (the patch number may differ from classmates on another OS — that is fine). It must **not** be 3.10 or 3.12+.

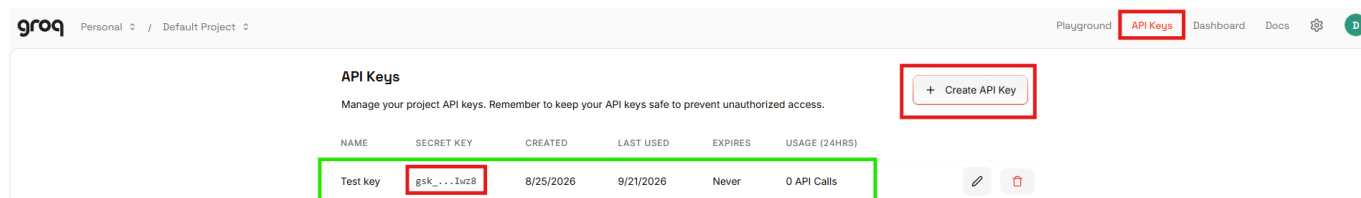
A5. API keys (Groq and Google Gemini)

The agentic part of Lab 1 needs at least **one** LLM provider; we recommend **both** Groq and Gemini so you can use multi-provider fallback in the main repo (see its [README.md](#) later).

Neither provider requires a payment method for the free tier used in this course.

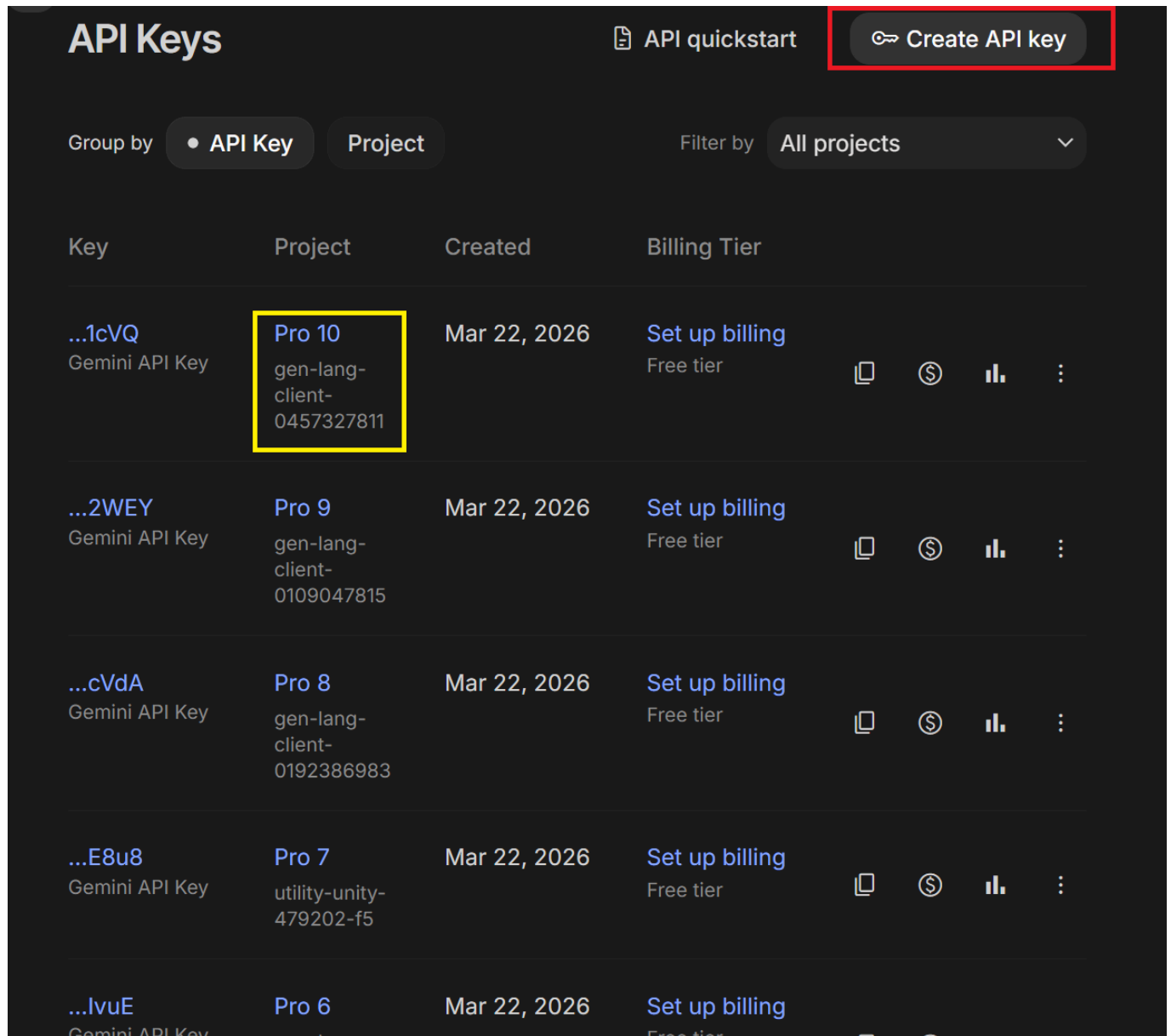
Groq (recommended primary)

1. Go to console.groq.com/keys, sign in, and click **Create API Key**.
2. Copy the key immediately — Groq only shows the full key once.
3. **One Groq key is enough**. Extra Groq keys under the same account do **not** increase your quota.



Google Gemini

1. Go to aistudio.google.com, sign in with a Google account.
2. Click **Get API key** (upper right), then **Create API key**.
3. Optional: create keys in **separate Google Cloud projects** (**Pro 1**, **Pro 2**, ...) if you want multiple keys for rotation later (`GOOGLE_API_KEY_1`, `_2`, ... in `.env`).



API Keys API quickstart Create API key

Group by API Key Project Filter by All projects

Key	Project	Created	Billing Tier	
...1cVQ Gemini API Key	Pro 10 gen-lang-client-0457327811	Mar 22, 2026	Set up billing Free tier	
...2WEY Gemini API Key	Pro 9 gen-lang-client-0109047815	Mar 22, 2026	Set up billing Free tier	
...cVdA Gemini API Key	Pro 8 gen-lang-client-0192386983	Mar 22, 2026	Set up billing Free tier	
...E8u8 Gemini API Key	Pro 7 utility-unity-479202-f5	Mar 22, 2026	Set up billing Free tier	
...lvuE Gemini API Key	Pro 6	Mar 22, 2026	Set up billing Free tier	

Create `config/.env`

From the repo root:

macOS / Linux:

```
cp config/.env.example config/.env
```

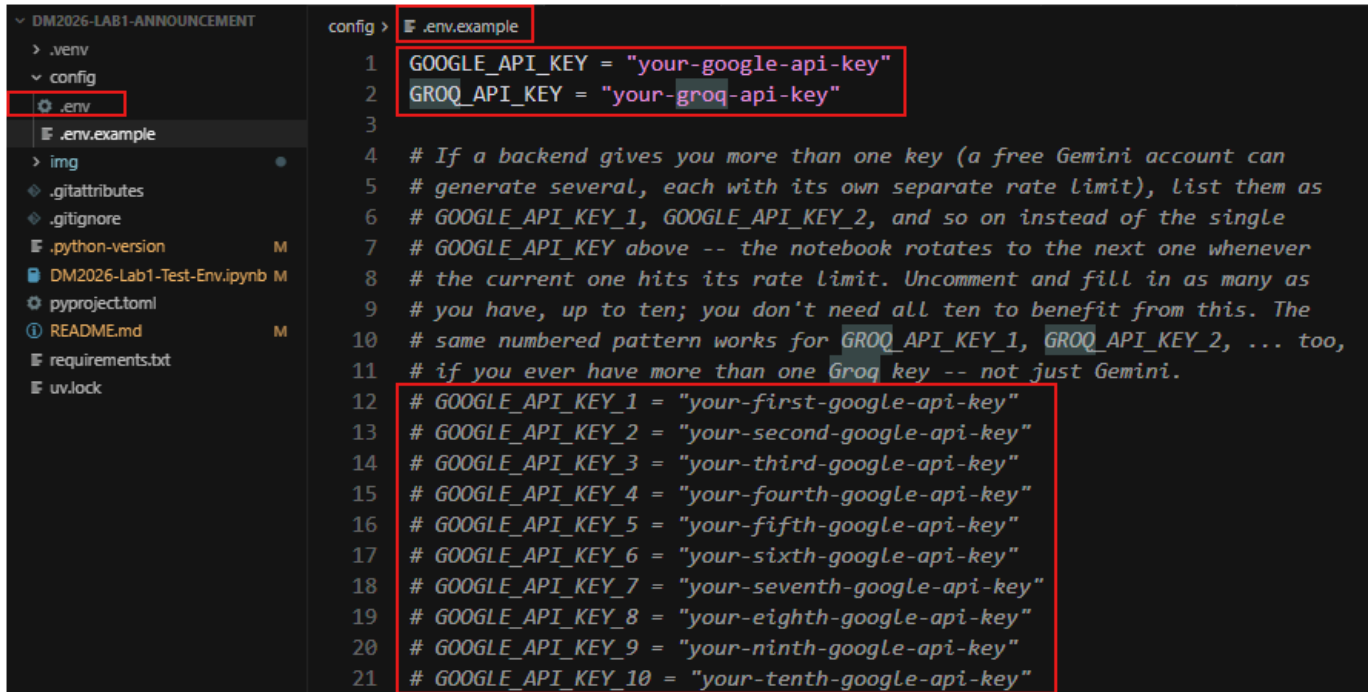
Windows (Command Prompt):

```
copy config\.env.example config\.env
```

Edit `config/.env` and paste your keys (replace the placeholder text):

```
GROQ_API_KEY="your-actual-groq-key"
GOOGLE_API_KEY="your-actual-google-key"
```

Do **not** commit `config/.env` or share keys publicly.

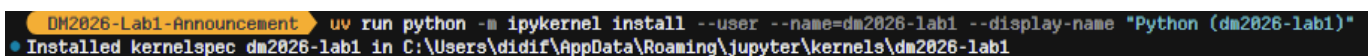


```
config > .env.example
1  GOOGLE_API_KEY = "your-google-api-key"
2  GROQ_API_KEY = "your-groq-api-key"
3
4  # If a backend gives you more than one key (a free Gemini account can
5  # generate several, each with its own separate rate limit), list them as
6  # GOOGLE_API_KEY_1, GOOGLE_API_KEY_2, and so on instead of the single
7  # GOOGLE_API_KEY above -- the notebook rotates to the next one whenever
8  # the current one hits its rate limit. Uncomment and fill in as many as
9  # you have, up to ten; you don't need all ten to benefit from this. The
10 # same numbered pattern works for GROQ_API_KEY_1, GROQ_API_KEY_2, ... too,
11 # if you ever have more than one Groq key -- not just Gemini.
12 # GOOGLE_API_KEY_1 = "your-first-google-api-key"
13 # GOOGLE_API_KEY_2 = "your-second-google-api-key"
14 # GOOGLE_API_KEY_3 = "your-third-google-api-key"
15 # GOOGLE_API_KEY_4 = "your-fourth-google-api-key"
16 # GOOGLE_API_KEY_5 = "your-fifth-google-api-key"
17 # GOOGLE_API_KEY_6 = "your-sixth-google-api-key"
18 # GOOGLE_API_KEY_7 = "your-seventh-google-api-key"
19 # GOOGLE_API_KEY_8 = "your-eighth-google-api-key"
20 # GOOGLE_API_KEY_9 = "your-ninth-google-api-key"
21 # GOOGLE_API_KEY_10 = "your-tenth-google-api-key"
```

A6. Register the Jupyter kernel

Still in the announcement repo root:

```
uv run python -m ipykernel install --user --name=dm2026-lab1 --display-name
"Python (dm2026-lab1)"
```



```
DM2026-Lab1-Announcement uv run python -m ipykernel install --user --name=dm2026-lab1 --display-name "Python (dm2026-lab1)"
• Installed kernelspec dm2026-lab1 in C:\Users\didif\AppData\Roaming\jupyter\kernels\dm2026-lab1
```

A7. Open notebooks locally

VS Code / Cursor / Antigravity

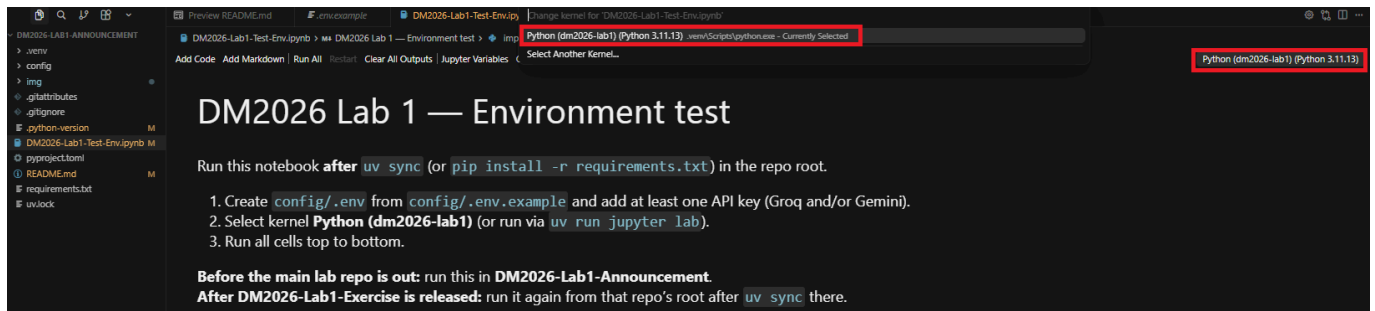
IMPORTANT NOTE: These programs are not so recommended due to some bugs observed in some environments when testing the Agent Chat Widget in the jupyter notebook, so it might also fail in your machine. You can try to use them, but it is under your own risk. **Jupyter Lab** has not presented any issues with the widget, so it is recommended to use it over this.

Open the folder of the repository from the **VS Code** / **Cursor** / **Antigravity** UI or execute the following commands in the terminal:

```
cd <path-to-DM2026-Lab1-Announcement>
```

code .

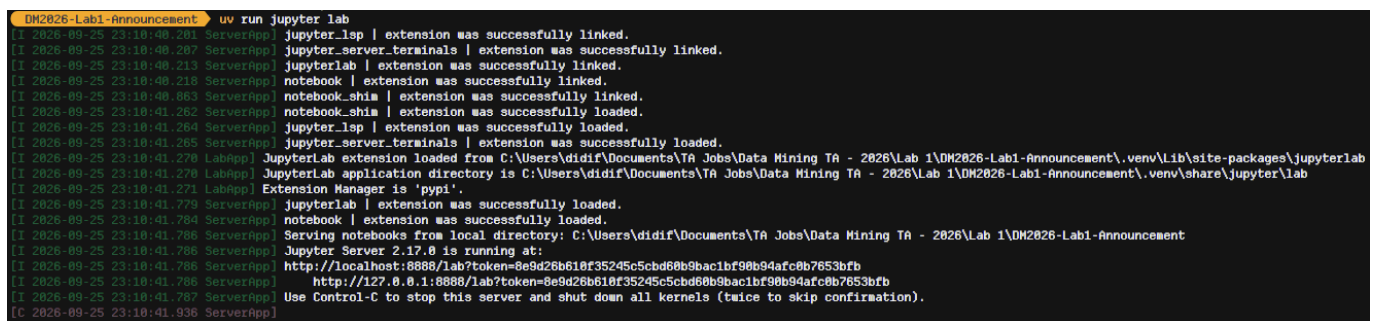
Open **DM2026-Lab1-Test-Env.ipynb** and choose kernel **Python (dm2026-lab1)** in the top-right corner.



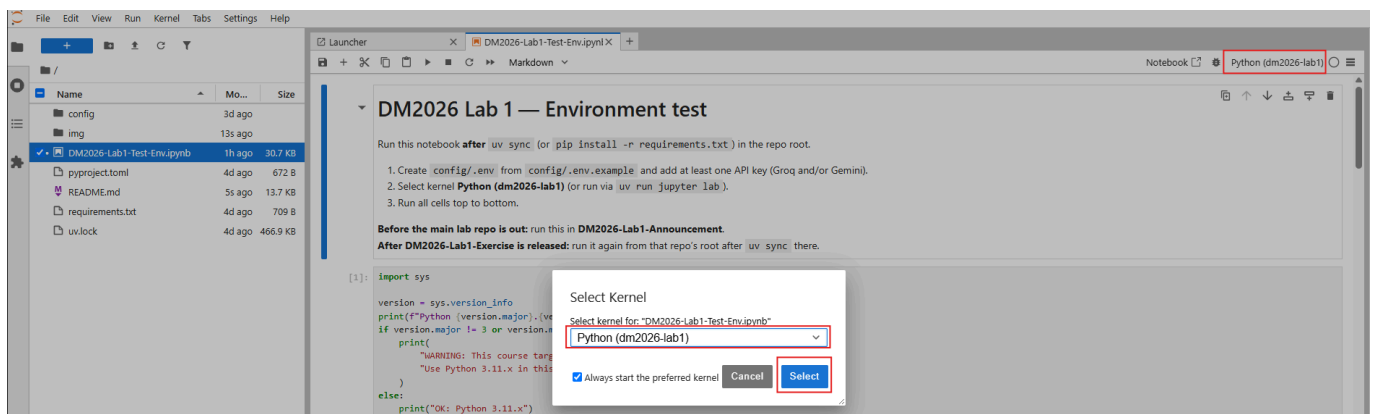
JupyterLab in the browser (recommended over classic Notebook for widgets)

IMPORTANT NOTE: Using **Jupyter Lab** for the **Agentic Section** of the lab is extremely recommended over some bugs that have been observed with VS Code / Cursor / Antigravity when running the widget to chat with the agent.

```
cd <path-to-DM2026-Lab1-Announcement>
uv run jupyter lab
```



Open **DM2026-Lab1-Test-Env.ipynb** and select **Python (dm2026-lab1)** as the kernel.

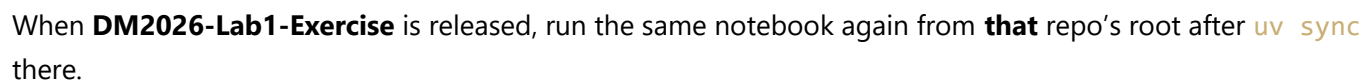
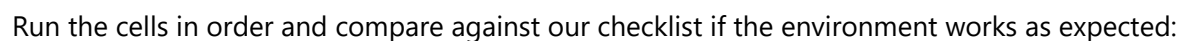


A8. Test your environment

Open **DM2026-Lab1-Test-Env.ipynb**, run **all cells** in order, and fix any errors before the lab session.

The notebook checks:

- The initial notebook provided already shows an example of how a successful environment setup looks like, first open it to take a look at it:



- **uv sync fails:** read the error, ensure you are in the repo root, run `uv python install 3.11` first, then `uv sync` again; try `uv lock` only if a TA tells you to — normally use the provided `uv.lock` as-is.
- **Wrong Python version:** run `uv python install 3.11`, delete `.venv` if needed, then `uv sync` from the repo root.
- **Wrong kernel:** the notebook must use **Python (dm2026-lab1)** from this project's `.venv` (`uv run jupyter lab` or VS Code with that interpreter).
- **Chat widget frozen later in the lab:** re-run `uv sync` in the **same** repo you use for notebooks; check `ipywidgets` is installed.
- **ModuleNotFoundError: PAMI:** you are not using the project environment — activate via `uv run` or select the correct kernel.

- **API tests fail:** check `config/.env`, no placeholder text left, keys not expired; try the other provider.

Ask classmates or TAs before the lab if you are stuck.

Best regards,
The TAs

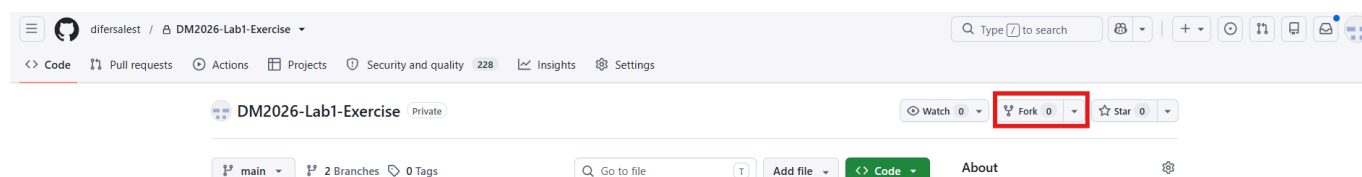
Part B — After DM2026-Lab1-Exercise is released

Do not start Part B until the main lab repository is announced on NTU COOL. Part A should already pass in the announcement repo; you will repeat the environment setup in the main repo, then do the lab and submit once.

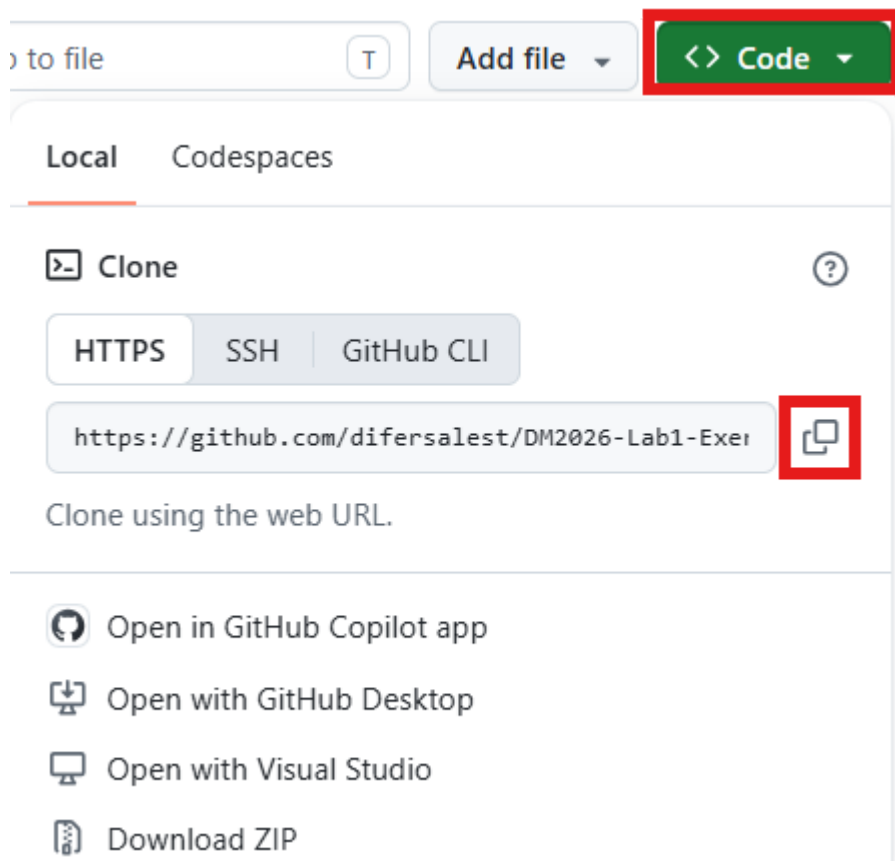
B1. Fork and clone the main lab repo

Go to the main lab repository on GitHub (link on NTU COOL when released starting **September 28 (Monday)** at **9:00 AM**), e.g. [DM2026-Lab1-Exercise](#).

Sign in, click **Fork** to copy it to your account.

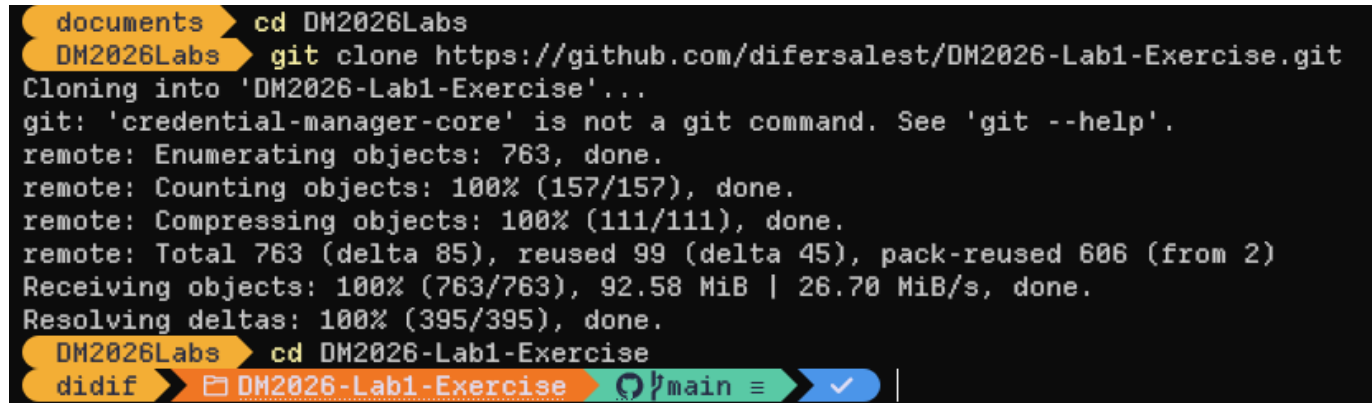


On your fork, click **Code** and copy the HTTPS URL.



Clone **your** fork (not the TA's upstream URL):

```
cd <yourpath>/DM2026Labs
git clone <your-fork-url>
cd DM2026-Lab1-Exercise
```



```
documents ➤ cd DM2026Labs
DM2026Labs ➤ git clone https://github.com/difersalest/DM2026-Lab1-Exercise.git
Cloning into 'DM2026-Lab1-Exercise'...
git: 'credential-manager-core' is not a git command. See 'git --help'.
remote: Enumerating objects: 763, done.
remote: Counting objects: 100% (157/157), done.
remote: Compressing objects: 100% (111/111), done.
remote: Total 763 (delta 85), reused 99 (delta 45), pack-reused 606 (from 2)
Receiving objects: 100% (763/763), 92.58 MiB | 26.70 MiB/s, done.
Resolving deltas: 100% (395/395), done.
DM2026Labs ➤ cd DM2026-Lab1-Exercise
didif ➤ DM2026-Lab1-Exercise 🔍 main ➤ ✓ |
```

B2. Environment in the main repo

From the **DM2026-Lab1-Exercise** root (same files as the announcement repo: `pyproject.toml`, `uv.lock`, etc.):

```
uv python install 3.11
uv sync
uv run python -m ipykernel install --user --name=dm2026-lab1 --display-name
"Python (dm2026-lab1)"
```

Copy or recreate `config/.env` with your API keys. Run **DM2026-Lab1-Test-Env.ipynb** again here to confirm.

Read **README.md** in this repo for the agentic exercise: work through **DM2026-Lab1-Master.ipynb** first, then **DM2026-Lab1-AgentDev.ipynb**, **DM2026-Lab1-AgenticPipeline.ipynb**, and **DM2026-Lab1-Homework.ipynb** in that order.

B3. Save progress with Git push

From the main repo root:

```
cd <path-to-your-DM2026-Lab1-Exercise>
git add .
git commit -m "your message"
git push
```

Use a meaningful commit message (e.g. **Finished Answering Master Questions 1–3**). Commit and push often.

```

Lab 1  cd DM2026-Lab1-Exercise
DM2026-Lab1-Exercise git add .
DM2026-Lab1-Exercise git commit -m "Updating README.md and .python-version"
• [main 933d611] Updating README.md and .python-version
  2 files changed, 5 insertions(+), 5 deletions(-)
DM2026-Lab1-Exercise git push
• git: 'credential-manager-core' is not a git command. See 'git --help'.
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 446 bytes | 446.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/difersalest/DM2026-Lab1-Exercise.git
98a5124..933d611  main -> main

```

B4. Submission (single deadline)

Submit on [NTU COOL](#) **before the deadline: October 19, 11:59 pm (Monday)**. There is **one** Lab 1 submission — include **all** of the following:

1. **GitHub repository link** — Copy and paste the link from your fork of **DM2026-Lab1-Exercise**, with your latest work pushed before the deadline (should include all the files cited below as well). Commits pushed after the deadline will have a **penalty** for the score. The **penalty** formula will be shared to you later on during the semester.
2. **A .zip file:** Name the **.zip** file in the following format
`{your_name_that_is_used_in_NTU_COOL}_{student_id}_files.zip`, check in NTU Cool how your name appears and use that to name the file in the specified format (e.g. 沙利葉_113065892_files.zip).

2.1 **The Solved Lab 1 Master Questions Document (PDF)** — complete the Master Questions document (provided on the main repository as a **.docx** word template), export to **PDF**, and upload it.

IMPORTANT NOTE: Change the name of the PDF in a similar fashion as the **.zip** file

`{your_name_that_is_used_in_NTU_COOL}_{student_id}_solved_master_questions.pdf` (e.g. 沙利葉_113065892_solved_master_questions.pdf).

2.2 **Agent conversation logs** — the three encrypted log files from the agentic notebooks (do **not** rename or edit them):

- `session_logs/agent_dev_session.jsonl.enc`
- `session_logs/agent_pipeline_session.jsonl.enc`
- `session_logs/homework_session.jsonl.enc`

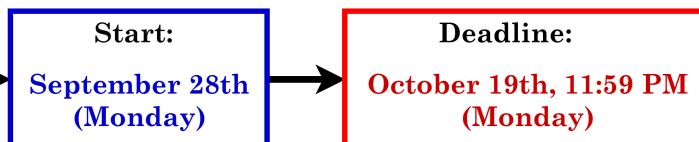
2.3 **AgenticPipeline & Homework Reports** - the two generated markdowns **.md** under the **reports/** folder with the written reports, and the **plots/** folder that the reports will reference to show the images generated by the tools from the agentic pipeline.

- `reports/{your_name_that_is_used_in_NTU_COOL}_{student_id}_agentic-pipeline_report.md`
- `reports/{your_name_that_is_used_in_NTU_COOL}_{student_id}_homework_report.md`
- `plots/`

ISA 5810 - DM2026 Lab 1

Deliverables in NTU Cool:

1. GitHub up-to-date repository link → Plain Text URL
2. Zip file: ➤ `{your_name_that_is_used_in_NTU_COOL}_{student_id}_files.zip`
 - 2.1 (PDF) Solved Master Notebook Questions
 - ➔ `{your_name_that_is_used_in_NTU_COOL}_{student_id}_solved_master_questions.pdf`
 - 2.2 (.jsonl.enc) Agent Conversation Logs
 - ``session_logs/agent_dev_session.jsonl.enc``
 - ``session_logs/agent_pipeline_session.jsonl.enc``
 - ``session_logs/homework_session.jsonl.enc``
 - 2.3 (.md + imgs): AgenticPipeline & Homework Reports
 - ``reports/{your_name_that_is_used_in_NTU_COOL}_{student_id}_agentic-pipeline_report.md``
 - ``reports/{your_name_that_is_used_in_NTU_COOL}_{student_id}_homework_report.md``
 - ``plots/``



On the assignment page, upload or paste each required item in the **Lab 1** section as instructed on NTU COOL.

To copy your GitHub repo link: GitHub → profile → **Your repositories** → your fork → copy the browser URL.

B5. Points Distribution:

1. **Solved Master Notebook Questions:** 20 pts.
2. **Solved Agent Dev Notebook:** 20 pts, graded from the encrypted conversation logs.
3. **Solved Agentic Pipeline Notebook:** Total 30 pts.
 - 3.1 **Guiding the Agent in the session:** 20 pts, graded from the encrypted conversation logs.
 - 3.2 **Agentic Pipeline Markdown Report:** 10 pts.
4. **Solved Homework Notebook:** Total 30 pts.
 - 4.1 **Guiding the Agent in the session:** 20 pts, graded from the encrypted conversation logs.
 - 4.2 **Homework Markdown Report:** 10 pts.

Total: 100 pts.

Again, as a reminder, **pushes made after the deadline will have points deducted from the final score according to a penalty formula.**

Good luck in the lab!